

# CS-405: Deep Learning

## Understanding Autoencoders and Variational Autoencoders

Dr. Sajjad Hussain

SEECS, NUST

March 24, 2026

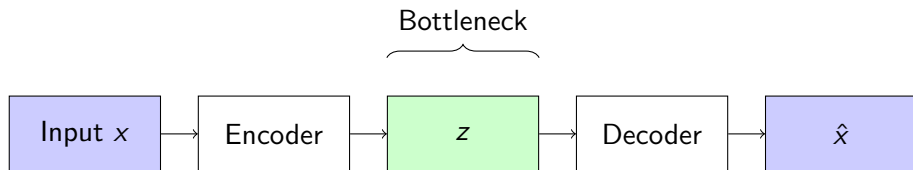
# Lecture Outline

- 1 Introduction to Autoencoders
- 2 Understanding Latent Space
- 3 Types of Autoencoders
- 4 Applications of Autoencoders
- 5 The Problem: Autoencoders Can't Generate
- 6 Variational Autoencoders (VAEs)

# What is an Autoencoder?

## Definition

An autoencoder is a neural network designed to learn efficient data representations **unsupervisedly** by learning to copy its input to its output.

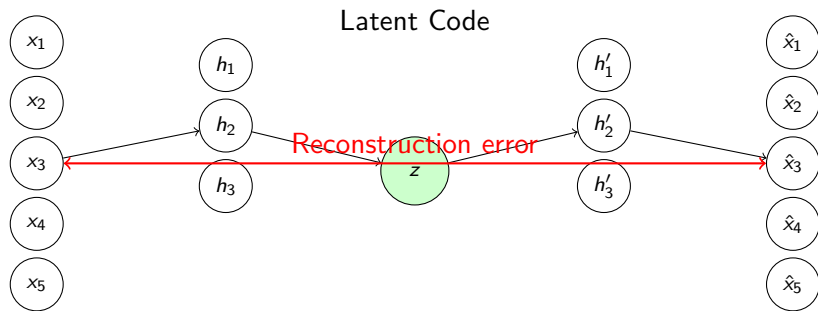


**Key insight:** The network is forced to learn a compressed representation  $z$  that preserves the most important information.

# Detailed Autoencoder Architecture

Input Layer    Encoder Hidden

Decoder Hidden    Output Layer



# How Autoencoders Work

**Encoder:** Compresses input

$$z = f_{\text{enc}}(x) = \sigma(W_e x + b_e)$$

**Decoder:** Reconstructs from code

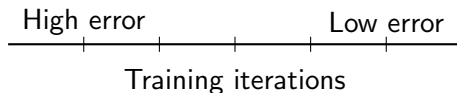
$$\hat{x} = f_{\text{dec}}(z) = \sigma(W_d z + b_d)$$

**Training Objective:**

$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

**Goal:** Minimize reconstruction error

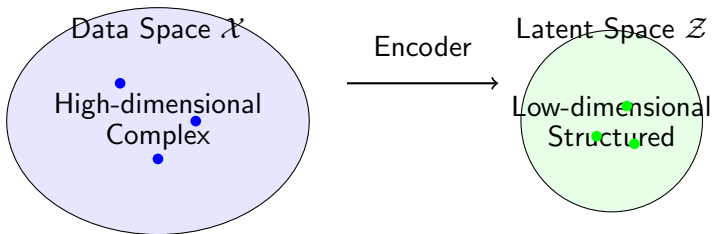
$$\min_{\theta} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2$$



# Understanding Latent Space

## Definition

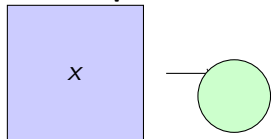
The latent space  $z$  is a compressed, lower-dimensional representation of the input data that captures its essential features.



**Analogy:** Like a ZIP file compresses a large document while preserving its essential content.

# Properties of Latent Space

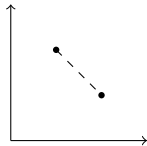
## Compactness



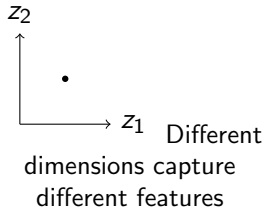
Compresses data to fewer dimensions

## Continuity

Similar points are close together



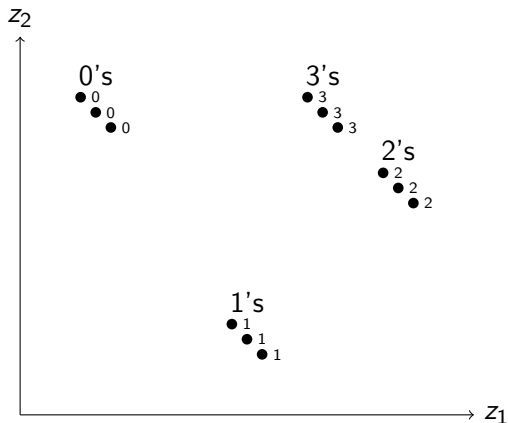
## Disentanglement



## Example

For face images:  $z_1$  might control smile,  $z_2$  might control age,  $z_3$  might control glasses.

# Visualizing Latent Space - MNIST Example

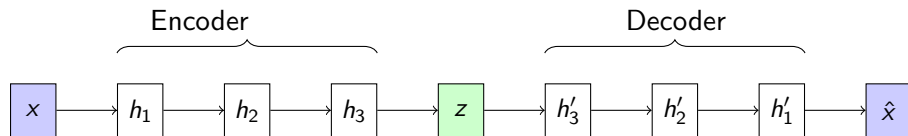


Each point = one digit image in latent space  
Similar digits cluster together



# Stacked (Deep) Autoencoders

**Idea:** Add multiple hidden layers for more expressive power

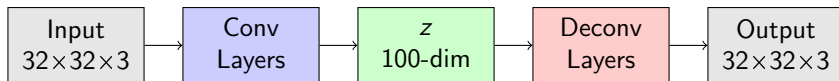


## Benefits:

- Can learn hierarchical features
- More complex representations
- Better reconstruction quality

# Convolutional Autoencoders (for Images)

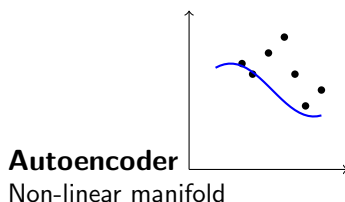
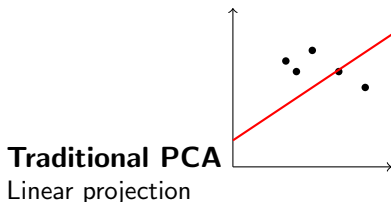
## Specialized architecture for image data



## How It Works

- **Encoder**: Convolutional layers progressively reduce spatial dimensions while increasing depth (channels)
- **Bottleneck**: Flattened into a compact latent vector  $z$  capturing essential features
- **Decoder**: Transposed convolutions upsample back to original dimensions
- **Result**: Learns efficient spatial representations without losing structural information

# Application 1: Dimensionality Reduction



## Advantages over PCA:

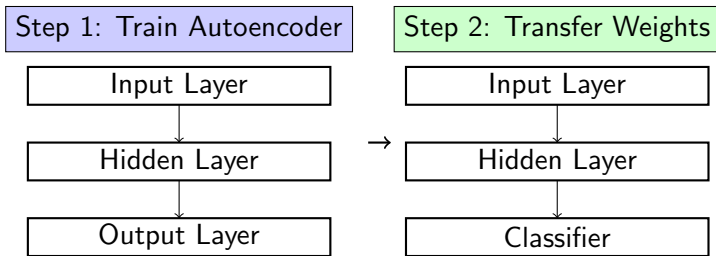
- Can learn non-linear transformations
- More flexible representations
- Can handle complex data structures

## Example

100×100 pixel image (10,000 dims) → 100-dim latent code (100× compression!)

# Application 2: Unsupervised Pre-training

**Historical importance:** Before deep learning took off, autoencoders helped train deep networks

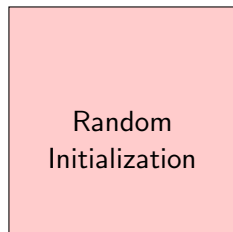


## Process:

- 1 Train autoencoder unsupervised on unlabeled data
- 2 Discard decoder, keep encoder weights
- 3 Add classifier on top, fine-tune with labels

# Why Unsupervised Pre-training Helps

## Without Pre-training



Hard to train

## With Pre-training



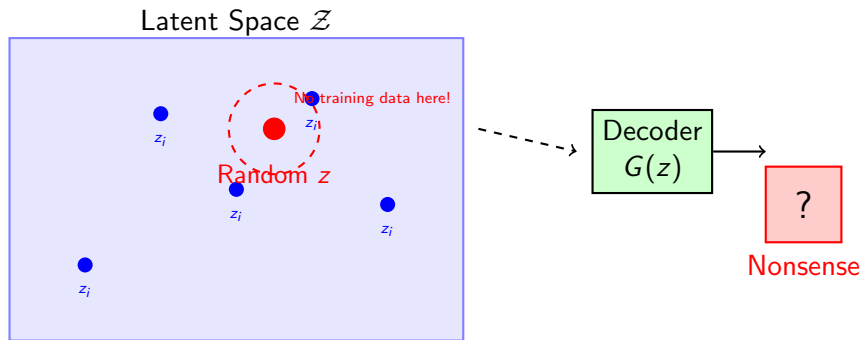
Easier fine-tuning

**Intuition:** The autoencoder has already learned useful features from the data distribution, giving the supervised network a "head start".

## Analogy

Like learning to recognize objects after already learning to see edges, shapes, and textures.

# The Problem: Autoencoders Can't Generate New Data



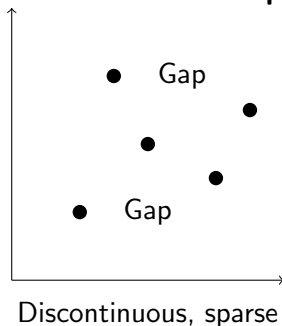
**Problem:** Random points in latent space map to meaningless outputs

## Why This Happens

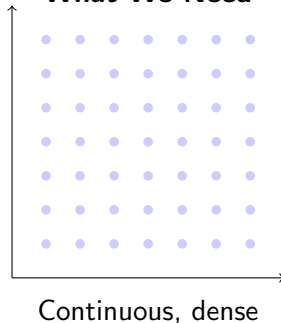
- Encoder maps each input to a **single point**
- Latent space has **disconnected regions**
- No constraint on the latent distribution
- Random sampling lands in **empty regions**

# Visualizing the Problem

Autoencoder Latent Space



What We Need



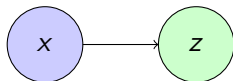
**Key insight:** For generation, we need the entire latent space to be meaningful, not just isolated points.



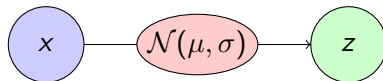
# Variational Autoencoders: The Solution

**Core Idea:** Make the encoder probabilistic, not deterministic.

Autoencoder:



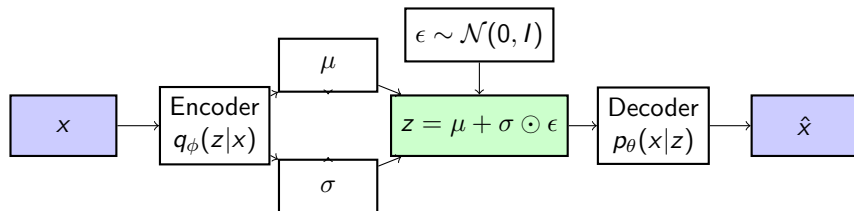
VAE:



**Instead of:**  $z = f(x)$  (single point)

**We use:**  $z \sim \mathcal{N}(\mu(x), \sigma^2(x))$  (probability distribution)

# VAE Architecture



## Key Components

- **Encoder:** Outputs distribution parameters  $\mu$  and  $\sigma$  (or  $\log(\sigma^2)$ ) as explained later)
- **Reparameterization:** Makes sampling differentiable  $z = \mu + \sigma \epsilon$
- **Decoder:** Reconstructs input from latent code
- **KL Divergence:** Regularizes latent space to  $\mathcal{N}(0, I)$

## Encoder

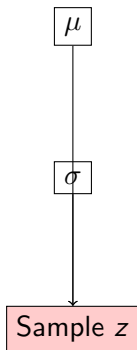
- Maps input  $x$  to distribution parameters
- Outputs mean  $\mu$  and variance  $\sigma^2$
- Represents uncertainty about latent code

## Decoder

- Maps sampled latent code  $z$  to reconstruction
- Learns  $p_{\theta}(x|z)$
- Can generate new data after training

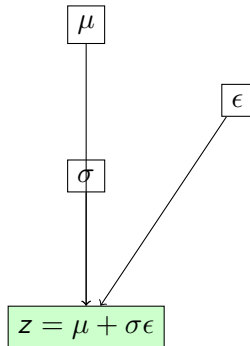
# The Reparameterization Trick

**Problem:** Gradients cannot backpropagate through random sampling!



Gradient stops here!

## Reparameterization Trick



Gradient flows through!

$$z = \mu + \sigma \odot \epsilon \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, I)$$

# VAE Loss Function

## The ELBO (Evidence Lower Bound) - Theoretical Form

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{z \sim q(z|x)} [\log p_{\theta}(x|z)]}_{\text{Expected Log-Likelihood}} - \underbrace{D_{\text{KL}}(q_{\phi}(z|x) \| p(z))}_{\text{KL Divergence}}$$

## Practical Implementation (for images)

$$\mathcal{L}_{\text{VAE}} = \underbrace{\text{MSE}(x, \hat{x})}_{\text{Reconstruction Loss}} + \underbrace{D_{\text{KL}}(q(z|x) \| \mathcal{N}(0, 1))}_{\text{Regularization}}$$

### Reconstruction Loss

- Makes output similar to input
- **MSE**:  $\|x - \hat{x}\|^2$  (for continuous data)
- **BCE** for binary data
- "Preserve the information"

### KL Divergence

- Regularizes latent space
- Forces  $q(z|x)$  close to prior  $\mathcal{N}(0, 1)$
- "Organize the space"

# What is KL Divergence?

## Simple Definition

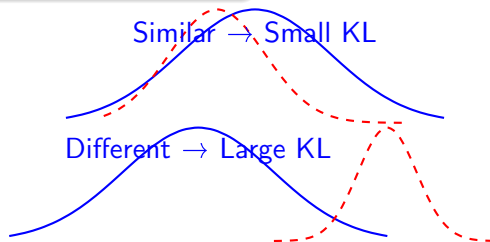
**KL Divergence** measures how much one probability distribution differs from another.

## What it tells us

- **Small KL** = distributions are similar
- **Large KL** = distributions are different

## In VAEs

- Compares  $q(z|x)$  to  $\mathcal{N}(0, 1)$
- Penalizes deviation from ideal
- Creates smooth latent space



# KL Divergence: The Formula

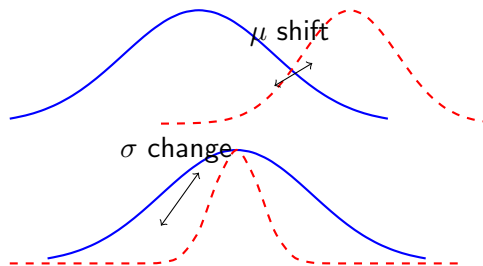
## For Gaussian Distributions

When  $q(z|x) = \mathcal{N}(\mu, \sigma^2)$  and  $p(z) = \mathcal{N}(0, 1)$ :

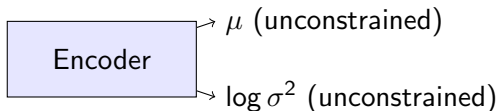
$$D_{KL} = -\frac{1}{2} \sum_{j=1}^d (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2)$$

### What each term does:

- $\mu^2$ : Penalizes shifting away from 0
- $\sigma^2$ : Penalizes variance  $\neq 1$
- $\log(\sigma^2)$ : Balances the penalty
- 1: Constant term



# Why Output $\log(\sigma^2)$ Instead of $\sigma$ ?



## The Problem with $\sigma^2$

- Variance  $\sigma^2$  must be  $\geq 0$
- Neural networks output unconstrained values
- Enforcing  $\sigma^2 \geq 0$  is tricky

## The Solution: $\log(\sigma^2)$

- $\log(\sigma^2)$  can be any real number
- No constraints needed!
- Recover  $\sigma^2 = \exp(\log(\sigma^2))$

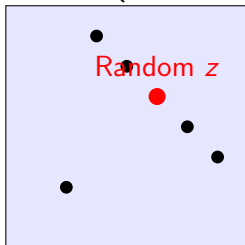
## Benefits

- **Numerical stability:** Avoids very small/large values
- **No constraints:** Network can output anything
- **Easy optimization:** Standard gradient descent works



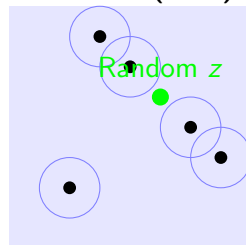
# How KL Divergence Shapes the Latent Space

## Without KL (Autoencoder)



Sparse, disconnected

## With KL (VAE)

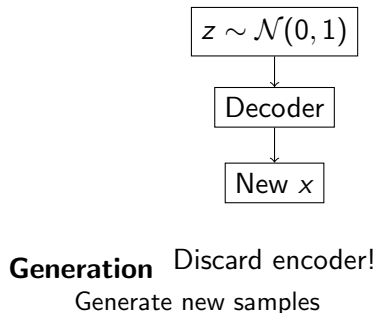
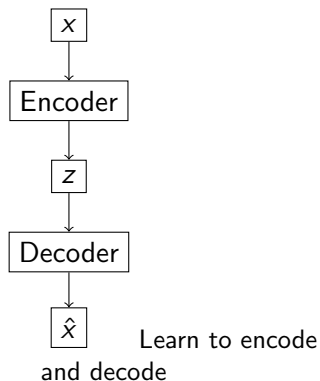


Smooth, continuous

## What KL Does During Training

- 1 **Pushes  $\mu \rightarrow 0$ :** Centers distributions at origin
- 2 **Pushes  $\sigma^2 \rightarrow 1$ :** Makes variances uniform
- 3 **Creates overlap:** Distributions from different inputs overlap
- 4 **Enables interpolation:** Any point in latent space is valid. i.e. Every point in latent space maps to a valid output!

# Training vs Generation in VAEs



# VAE vs Autoencoder: Key Differences

| Property       | Autoencoder      | VAE                                     |
|----------------|------------------|-----------------------------------------|
| Encoder output | Single point $z$ | Distribution $\mathcal{N}(\mu, \sigma)$ |
| Latent space   | Sparse           | Continuous                              |
| Can generate?  | No               | Yes                                     |
| Interpolation  | Not smooth       | Smooth                                  |
| Regularization | None             | KL divergence                           |

## Analogy:

- AE: Remembering exact addresses of friends
- VAE: Understanding the city layout - can find new places

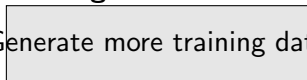
# Applications of VAEs

## Image Generation



## Data Augmentation

Generate more training data



## Anomaly Detection



Anomaly



## Drug Discovery

Generate new molecules

# Summary: Autoencoders

## What We Learned

- Autoencoders learn compressed representations via reconstruction
- Latent space captures essential features
- Stacked and convolutional variants for complex data
- Applications: dimensionality reduction, pre-training
- **Limitation:** Cannot generate new data

# Summary: Variational Autoencoders

## Key Takeaways

- VAEs make the encoder probabilistic
- Reparameterization trick enables training
- KL divergence regularizes latent space
- Results in smooth, continuous latent space
- **Can generate new data!**

VAE = Autoencoder + Probability + Regularization

Encoder:  $x \rightarrow \mathcal{N}(\mu, \sigma)$

Decoder:  $z \rightarrow \hat{x}$

# References and Further Reading



D. Kingma and M. Welling (2013).  
Auto-Encoding Variational Bayes.  
*ICLR*.



C. Doersch (2016).  
Tutorial on Variational Autoencoders.  
*arXiv:1606.05908*.



I. Goodfellow, Y. Bengio, and A. Courville (2016).  
Deep Learning.  
*MIT Press* (Chapter 14).



D. Rumelhart, G. Hinton, and R. Williams (1985).  
Learning internal representations by error propagation.  
*PDP Group*.